

✓ Low code Exploratory Data Tools

Created: 03/15/2023

Updated: 03/17/2023

(Must be executed again in Google Colab, to show all outputs).

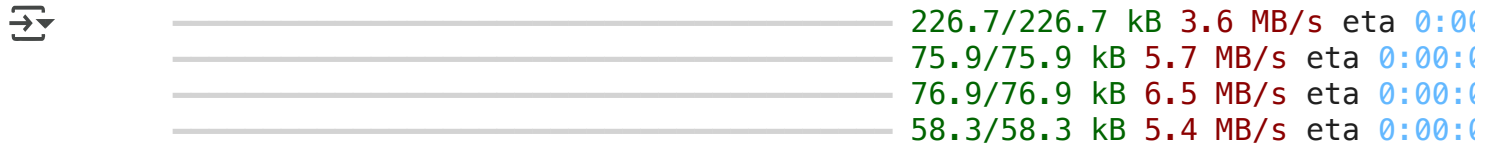
This Google Colab Jupyter Notebook shows EDA examples using a set of diverse low-code exploratory data tools:

- [ydata-profiling](#), also known as `pandas-profiling`, the primary goal is to provide a one-line Exploratory Data Analysis (EDA) experience in a consistent and fast solution.
- [sweetviz](#). Sweetviz is an open-source Python library that generates beautiful, high-density visualizations to kickstart EDA (Exploratory Data Analysis) with just two lines of code. Output is a fully self-contained HTML application.
- [lux APLI](#). Lux is a Python library that facilitate fast and easy data exploration by automating the visualization and data analysis process. By simply printing out a dataframe in a Jupyter notebook, Lux recommends a set of visualizations highlighting interesting trends and patterns in the dataset.
- [DataPrep](#). DataPrep.EDA is the fastest and the easiest EDA (Exploratory Data Analysis) tool in Python. It allows you to understand a Pandas/Dask DataFrame with a few lines of code in seconds.
- [AutoViz](#). AutoViz performs automatic visualization of any dataset with one line of code. Give it any input file (CSV, txt or json format) of any size and AutoViz will visualize it.

✓ ydata-profiling example.

```
#!pip install visions==0.7.4 --quiet
```

```
!pip install openai --quiet
```



A progress bar for the installation of the 'openai' package. It consists of four horizontal bars, each representing a different component. The first bar is at 100% (226.7/226.7 kB) with a speed of 3.6 MB/s. The second bar is at 100% (75.9/75.9 kB) with a speed of 5.7 MB/s. The third bar is at 100% (76.9/76.9 kB) with a speed of 6.5 MB/s. The fourth bar is at 100% (58.3/58.3 kB) with a speed of 5.4 MB/s. All bars are green, indicating successful completion. The estimated time remaining (eta) for each is 0:00:00.

Component	Downloaded	Size	Speed	ETA
1	226.7/226.7 kB	226.7 kB	3.6 MB/s	0:00:00
2	75.9/75.9 kB	75.9 kB	5.7 MB/s	0:00:00
3	76.9/76.9 kB	76.9 kB	6.5 MB/s	0:00:00
4	58.3/58.3 kB	58.3 kB	5.4 MB/s	0:00:00

```
!pip install tiktoken --quiet
```

```
!pip install cohere --quiet
```

```
!pip install fastapi --quiet
```



A progress bar for the installation of the 'fastapi' package. It consists of two horizontal bars. The first bar is at 100% (92.1/92.1 kB) with a speed of 2.3 MB/s. The second bar is at 100% (71.5/71.5 kB) with a speed of 4.9 MB/s. Both bars are green, indicating successful completion. The estimated time remaining (eta) for each is 0:00:00.

Component	Downloaded	Size	Speed	ETA
1	92.1/92.1 kB	92.1 kB	2.3 MB/s	0:00:00
2	71.5/71.5 kB	71.5 kB	4.9 MB/s	0:00:00

```
!pip install python-multipart --quiet
```

```
!pip install uvicorn --quiet
```

```
!pip install kaleido --quiet
```

```
# pandas-profiling requirements
```

```
import sys
!{sys.executable} -m pip install -U ydata-profiling[notebook] --quiet
!jupyter nbextension enable --py widgetsnbextension
```

```


  _____ 357.8/357.8 kB 3.9 MB/s eta 0:00
  _____ 102.7/102.7 kB 4.5 MB/s eta 0:00
Preparing metadata (setup.py) ... done
  _____ 686.1/686.1 kB 7.3 MB/s eta 0:00
  _____ 293.3/293.3 kB 7.8 MB/s eta 0:00
  _____ 296.5/296.5 kB 7.6 MB/s eta 0:00
  _____ 4.7/4.7 MB 18.3 MB/s eta 0:00:00
  _____ 1.6/1.6 MB 29.9 MB/s eta 0:00:00
Building wheel for htmlmin (setup.py) ... done
Enabling notebook extension jupyter-js-widgets/extension...
Paths used for configuration of notebook:
    /root/.jupyter/nbconfig/notebook.json
Paths used for configuration of notebook:

    - Validating: OK
Paths used for configuration of notebook:
    /root/.jupyter/nbconfig/notebook.json

```

```
import numpy as np
import pandas as pd
from ydata_profiling import ProfileReport
```

```
# Read Penguins dataset
```

```
pdataset = "https://raw.githubusercontent.com/clizarraga-UAD7/Datasets/main/pengu
df = pd.read_csv(pdataset)
```

```
# General dataframe info
df.info()
```

```
↗ <class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species                344 non-null    object
1   island                 344 non-null    object
2   culmen_length_mm       342 non-null    float64
3   culmen_depth_mm       342 non-null    float64
4   flipper_length_mm     342 non-null    float64
5   body_mass_g            342 non-null    float64
6   sex                   334 non-null    object
dtypes: float64(4), object(3)
memory usage: 18.9+ KB
```

```
# Drop rows with missing numbers
df=df.dropna()
df.info()
```

```
↗ <class 'pandas.core.frame.DataFrame'>
Int64Index: 334 entries, 0 to 343
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species                334 non-null    object
1   island                 334 non-null    object
2   culmen_length_mm       334 non-null    float64
3   culmen_depth_mm       334 non-null    float64
4   flipper_length_mm     334 non-null    float64
5   body_mass_g            334 non-null    float64
6   sex                   334 non-null    object
dtypes: float64(4), object(3)
memory usage: 20.9+ KB
```

```
# Pandas df.describe() summary
df.describe()
```



	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g
count	334.000000	334.000000	334.000000	334.000000
mean	43.994311	17.160479	201.014970	4209.056886
std	5.460521	1.967909	14.022175	804.836129
min	32.100000	13.100000	172.000000	2700.000000
25%	39.500000	15.600000	190.000000	3550.000000
50%	44.500000	17.300000	197.000000	4050.000000
75%	48.575000	18.700000	213.000000	4793.750000
max	59.600000	21.500000	231.000000	6300.000000

```
# To generate the standard profiling report
```

```
profile = ProfileReport(df, title="Profiling Report")
```

```
# displaying the report as a set of widgets
```

```
profile.to_widgets()
```



```
/usr/local/lib/python3.10/dist-packages/ydata_profiling/profile_report.py:52:
  warnings.warn(
```

```
Summarize dataset: 32/32 [00:07<00:00, 3.79it/s,
100% Completed]
```

```
Generate report structure: 1/1 [00:04<00:00,
100% 4.71s/it]
```

Overview	Variables	Interactions	Correlations	Missing values	Sample
	Variables	species			

species

island

island	Distinct	3	Biscoe	1...
Categorical			Dream	1...
HIGH CORRELATION	Distinct (%)	0.9%	Torgersen	4.
	Missing	0		
	Missing (%)	0.0%		
	Memory size	5.2 KiB		

More details

Overview		Categories		Words		Characters	
Max length	9	Total characters	2022	Unique	0	1st row	Torgersen
Median length	6	Distinct characters	13	Unique (%)	0.0%	2nd row	Torgersen
Mean length	6.0538922	Distinct categories	2			3rd row	Torgersen
Min length	5	Distinct scripts	1			4th row	Torgersen
		Distinct blocks	1			5th row	Torgersen
The Unicode Standard assigns character properties							

```
# The HTML report can be directly embedded in a cell  
  
profile.to_notebook_iframe()
```



Render HTML: 100%

1/1 [00:01<00:00, 1.40s/it]

Overview

Dataset statistics

Number of variables	7
Number of observations	334
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	20.9 KiB
Average record size in memory	64.0 B

Variable types

Categorical	3
Numeric	4

Alerts

body_mass_g is highly overall correlated with culmen_length_mm and 2 other fields (culmen_length_mm, flipper_length_mm, species)	High correlation
culmen_depth_mm is highly overall correlated with flipper_length_mm and 1 other fields (flipper_length_mm, species)	High correlation

✓ Sweetviz example.

```
# Sweetviz install
!pip install sweetviz --quiet
```

 15.1/15.1 MB 27.2 MB/s eta 0:00:00

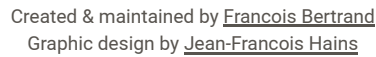
```
import sweetviz as sv

my_dataframe = df.copy()

my_report = sv.analyze(df)
my_report.show_html() # Default arguments will generate to "SWEETVIZ_REPORT.html"
```

 Done! Use 'show' commands to display/save. [100%] 00:00 -> (00:00 left)
Report SWEETVIZ REPORT.html was generated! NOTEBOOK/COLAB USERS: the web browser

Note: Google Colab does not open HTML. Please download the HTML to your local computer and open it with a browser.



334
0
84.7 kb
7
3
4
0

DataFrame

RANGE	8.4
IQR	3.1

✓ Lux API example.

```
# Lux installation
```

```
!pip install lux-api --quiet
```

```

127.0/127.0 kB 2.8 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
2.5/2.5 MB 34.2 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
45.3/45.3 kB 4.0 MB/s eta 0:00:00
Building wheel for lux-api (pyproject.toml) ... done
Building wheel for lux-widget (setup.py) ... done

```

```
# Activate Jupyter notebook extension
```

```
!jupyter nbextension install --py luxwidget
```

```
!jupyter nbextension enable --py luxwidget
```

```

Installing /usr/local/lib/python3.10/dist-packages/luxwidget/nbextension/stat:
Making directory: /usr/local/share/jupyter/nbextensions/luxwidget/
Copying: /usr/local/lib/python3.10/dist-packages/luxwidget/nbextension/static,
Copying: /usr/local/lib/python3.10/dist-packages/luxwidget/nbextension/static,
Copying: /usr/local/lib/python3.10/dist-packages/luxwidget/nbextension/static,
- Validating: OK

```

To initialize this nbextension in the browser every time the notebook (or

```
jupyter nbextension enable luxwidget --py
```

```
Enabling notebook extension luxwidget/extension...
```

```
Paths used for configuration of notebook:
```

```
  /root/.jupyter/nbconfig/notebook.json
```

```
Paths used for configuration of notebook:
```

```
  - Validating: OK
```

```
Paths used for configuration of notebook:
```

```
  /root/.jupyter/nbconfig/notebook.json
```

```
# Load Lux and add local commands for Google Colab
```

```
import lux
import pandas as pd
```

```
from google.colab import output
output.enable_custom_widget_manager()
```

```
# Read dataset
```

```
pdataset = "https://raw.githubusercontent.com/clizarraga-UAD7/Datasets/main/pengu"
df = pd.read_csv(pdataset)
```

```
# Drop rows with missing numbers
df=df.dropna()
```

```
# Now print df and a Toggle widget will appear to switch between Pandas and Lux
```

```
df
```



Toggle Pandas/Lux



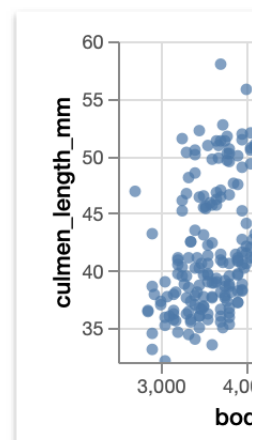
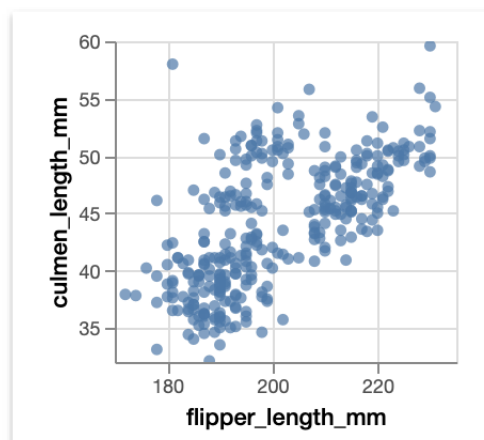
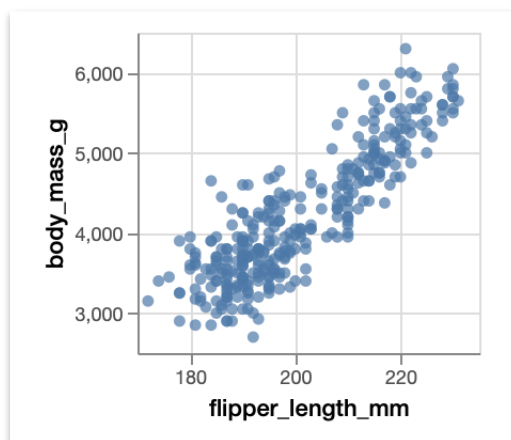
Correlation

Distribution

Occurrence



Show relationships between two quantitative attributes.



✓ DataPrep example

```
!pip install -U git+https://github.com/sfu-db/dataprep.git@develop --quiet
```

```

⇨ Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
_____ 18.5/18.5 MB 37.4 MB/s eta 0:00:00
_____ 133.6/133.6 kB 14.3 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
_____ 3.1/3.1 MB 41.4 MB/s eta 0:00:00
_____ 1.0/1.0 MB 43.1 MB/s eta 0:00:00
_____ 1.0/1.0 MB 42.1 MB/s eta 0:00:00
_____ 3.0/3.0 MB 62.0 MB/s eta 0:00:00
_____ 764.0/764.0 kB 16.9 MB/s eta 0:00:00
_____ 6.4/6.4 MB 51.8 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
_____ 49.6/49.6 kB 4.9 MB/s eta 0:00:00
Building wheel for dataprep (pyproject.toml) ... done
Building wheel for sqlalchemy (setup.py) ... done
Building wheel for metaphone (setup.py) ... done
ERROR: pip's dependency resolver does not currently take into account all the
bigframes 0.20.1 requires sqlalchemy<3.0dev,>=1.4, but you have sqlalchemy 1.3.24
ipython-sql 0.5.0 requires sqlalchemy>=2.0, but you have sqlalchemy 1.3.24 whi
panel 1.3.8 requires bokeh<3.4.0,>=3.2.0, but you have bokeh 2.4.3 which is in
tiktoken 0.6.0 requires regex>=2022.1.18, but you have regex 2021.11.10 which
ydata-profiling 4.6.4 requires pydantic>=2, but you have pydantic 1.10.14 whi

```

```
from dataprep.eda import *
from dataprep.eda import plot, plot_correlation, plot_missing, plot_diff, create_
```

 -----

```
ModuleNotFoundError                                Traceback (most recent call last)
<ipython-input-31-57b7790d8da8> in <cell line: 1>()
----> 1 from dataprep.eda import *
      2 from dataprep.eda import plot, plot_correlation, plot_missing,
      plot_diff, create_report

----- 7 frames -----
/usr/local/lib/python3.10/dist-packages/pydantic/_migration.py in
wrapper(name)

ModuleNotFoundError: No module named 'pydantic. internal'
```

```
# Read dataset
```

```
pdataset = "https://raw.githubusercontent.com/clizarraga-UAD7/Datasets/main/pengu
df = pd.read_csv(pdataset)
```

```
# Drop rows with missing numbers
#df=df.dropna()
```

```
plot(df)
```

```
# Plot missing values
```

```
plot_missing(df)
```

```
# Missing value overview
```

```
plot_missing(df, "sex")
```

```
# Correlation overview
```

```
plot_correlation(df)
```

```
# Understand how other columns correlated to the given column
```

```
plot_correlation(df, "body_mass_g")
```

```
# Numerical column overview
```

```
plot(df, "body_mass_g")
```

```
# Categorical value overview
```

```
plot(df, "sex")
```

```
# Numerical vs. Numerical column relationship
```

```
plot(df, "flipper_length_mm", "body_mass_g")
```

```
# Numerical vs. Categorical column relationship
```

```
plot(df, "body_mass_g", "sex")
```

```
# Create a full report
```

```
create_report(df)
```

✓ AutoViz example

```
# Install AutoViz
```

```
!pip install autoviz --quiet
```



```
68.3/68.3 kB 2.2 MB/s eta 0:00:00
421.5/421.5 kB 6.2 MB/s eta 0:00:00
4.3/4.3 MB 13.5 MB/s eta 0:00:00
3.1/3.1 MB 26.4 MB/s eta 0:00:00
17.3/17.3 MB 35.9 MB/s eta 0:00:00
1.9/1.9 MB 66.4 MB/s eta 0:00:00
255.9/255.9 MB 4.5 MB/s eta 0:00:00
87.3/87.3 kB 9.6 MB/s eta 0:00:00
20.8/20.8 MB 54.6 MB/s eta 0:00:00
20.8/20.8 MB 56.7 MB/s eta 0:00:00
20.8/20.8 MB 61.1 MB/s eta 0:00:00
20.1/20.1 MB 18.4 MB/s eta 0:00:00
20.1/20.1 MB 50.5 MB/s eta 0:00:00
20.4/20.4 MB 65.6 MB/s eta 0:00:00
20.4/20.4 MB 44.8 MB/s eta 0:00:00
20.1/20.1 MB 13.5 MB/s eta 0:00:00
20.0/20.0 MB 31.8 MB/s eta 0:00:00
20.0/20.0 MB 38.5 MB/s eta 0:00:00
20.0/20.0 MB 49.3 MB/s eta 0:00:00
20.0/20.0 MB 15.3 MB/s eta 0:00:00
20.0/20.0 MB 44.5 MB/s eta 0:00:00
20.0/20.0 MB 47.1 MB/s eta 0:00:00
19.9/19.9 MB 15.6 MB/s eta 0:00:00
19.9/19.9 MB 48.3 MB/s eta 0:00:00
19.9/19.9 MB 48.2 MB/s eta 0:00:00
19.9/19.9 MB 16.9 MB/s eta 0:00:00
20.8/20.8 MB 41.8 MB/s eta 0:00:00
```

```
ERROR: pip's dependency resolver does not currently take into account all the
ydata-profiling 4.6.4 requires pydantic>=2, but you have pydantic 1.10.14 which
```



```
# Load AutoViz libraries
```

```
from autoviz.AutoViz_Class import AutoViz_Class
AV = AutoViz_Class()
```

```
# AutoViz does not display plots automatically. The next line is needed.
%matplotlib inline
```

➞ Imported v0.1.804. After importing autoviz, you must run '%matplotlib inline'

```
AV = AutoViz_Class()
dfte = AV.AutoViz(filename, sep=',', depVar='', dfte=None, header=0, verbose=1,
                  chart_format='svg', max_rows_analyzed=150000, max_cols_analyzed=150000)
```

```
# Generate visualizations
```

```
pdataset = "https://raw.githubusercontent.com/clizarraga-UAD7/Datasets/main/pengu"
```

```
AV.AutoViz(pdataset)
```

➞ Shape of your Data Set loaded: (344, 7)

```
#####
##### C L A S S I F Y I N G   V A R I A B L E S #####
#####
Classifying variables in data set...
Number of Numeric Columns = 4
Number of Integer-Categorical Columns = 0
Number of String-Categorical Columns = 3
Number of Factor-Categorical Columns = 0
Number of String-Boolean Columns = 0
Number of Numeric-Boolean Columns = 0
Number of Discrete String Columns = 0
Number of NLP String Columns = 0
Number of Date Time Columns = 0
Number of ID Columns = 0
Number of Columns to Delete = 0
7 Predictors classified...
No variables removed since no ID or low-information variables found in
To fix these data quality issues in the dataset, import FixDQ from autoviz...
All variables classified into correct types.
```

	Data Type	Missing Values%	Unique Values%	Minimum Value	Maximum Value	DQ Issue
species	object	0.000000	0			No issue
island	object	0.000000	0			No issue

2 missing values.

them into a single category or drop the categories., Mixed dtypes: has 2 different data types: object, float,

Pair-wise Scatter Plot of all Continuous Variables

